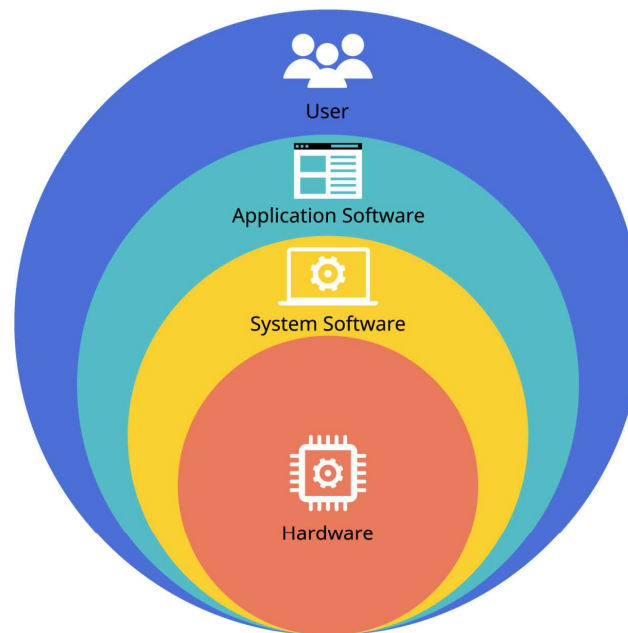


Understanding Software Architecture and Emerging Trends

Software Types, Design Patterns,
MVC, SOA & Microservices

System Software vs Application Software

- System software controls hardware and manages system resources.
- It provides a platform on which application software can run.
- Application software is designed to help users perform specific tasks.
- Examples include browsers, text editors, and media players.

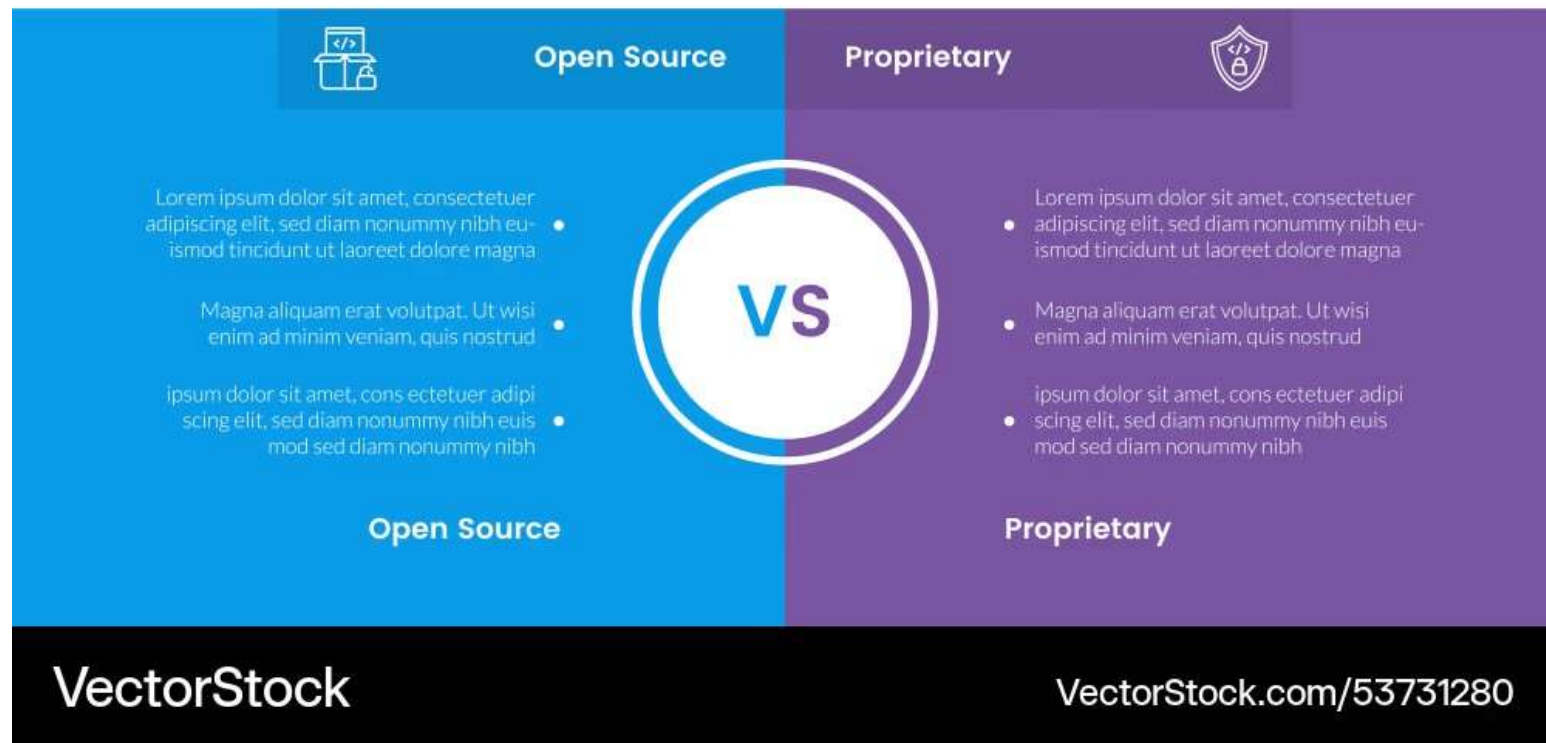


Proprietary, Open-Source, and Freeware

- Proprietary software is owned by a company and source code is restricted.
- Open-source software allows users to view, modify, and distribute code.
- Freeware is available at no cost but usually without source code access.

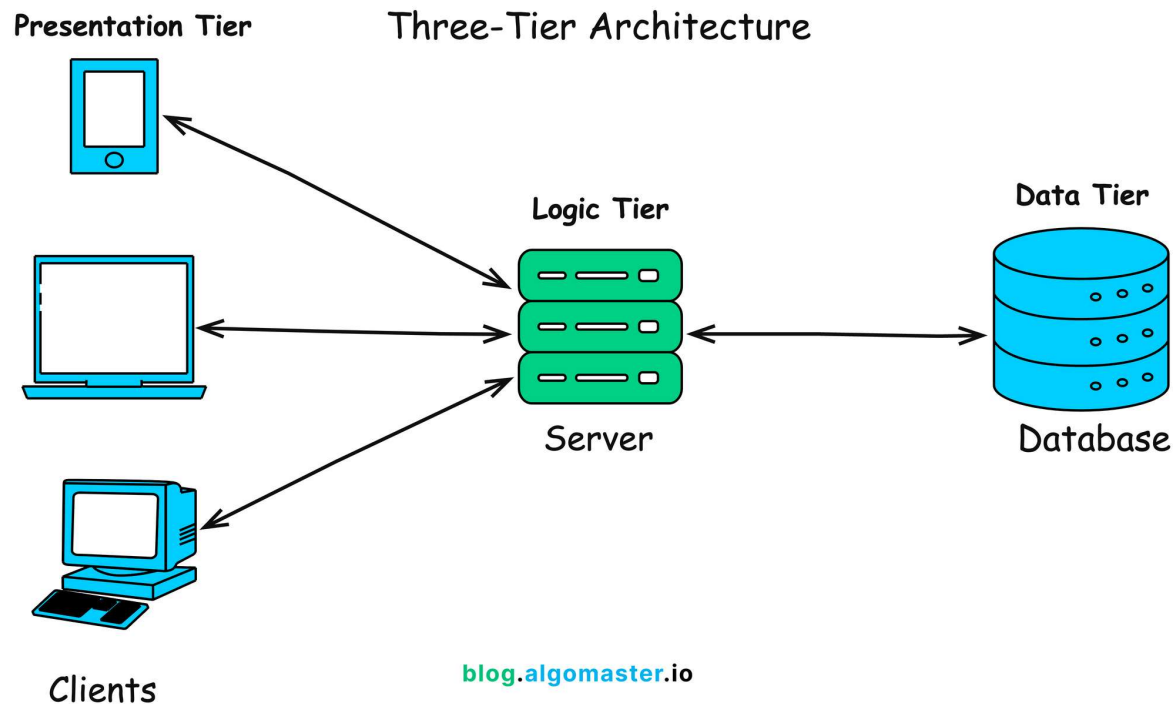
Open Source vs Proprietary

Lorem ipsum dolor sit amet, consectetur adipiscing elit, sed do eiusmod tempor incididunt ut labore et dolore magna aliqua. Duis aute irure dolor in reprehenderit in voluptate velit esse sed do



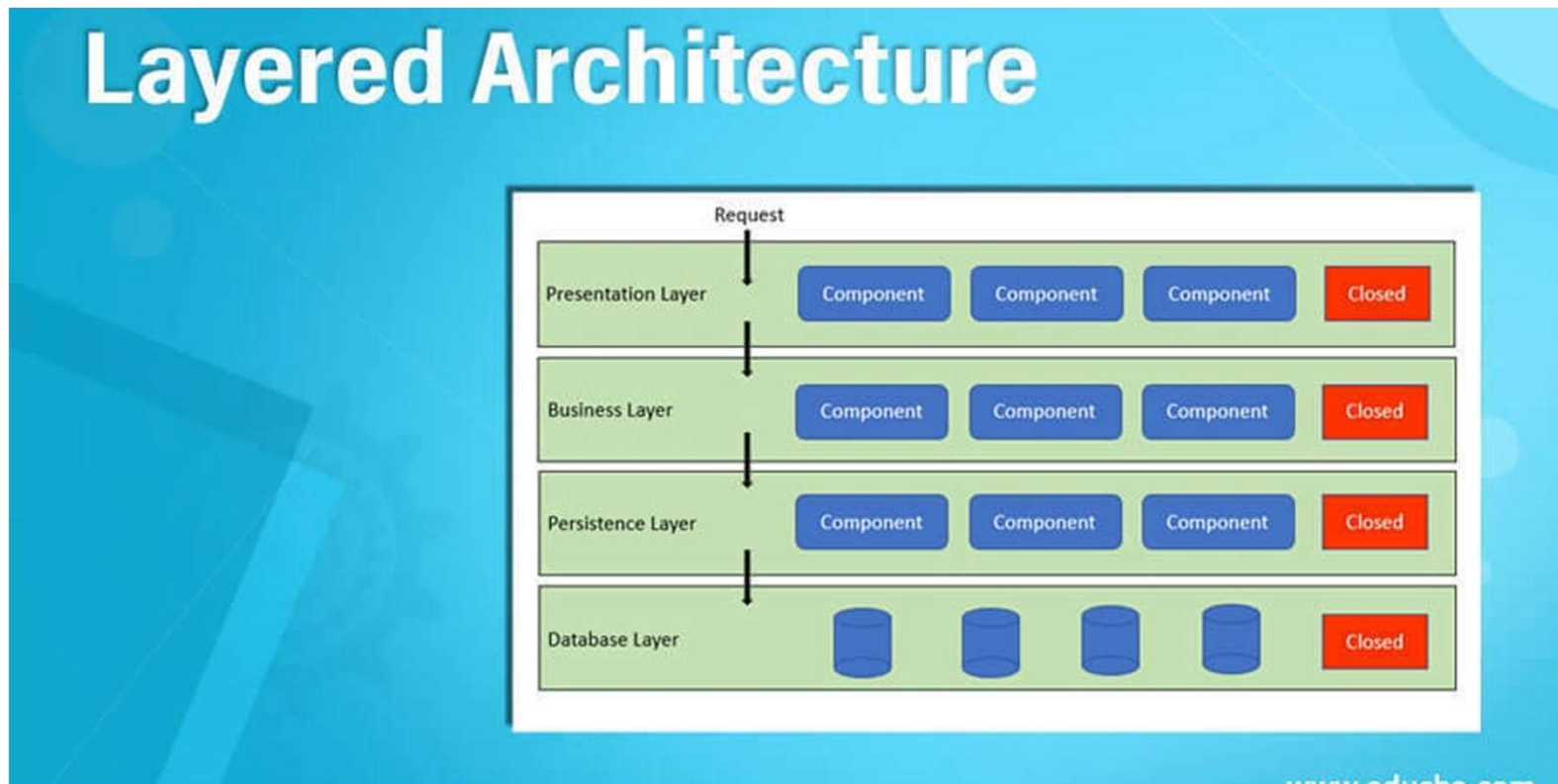
Web, Desktop, and Mobile Applications

- Web applications run inside browsers and are platform-independent.
- Desktop applications are installed locally and OS-dependent.
- Mobile applications are designed specifically for smartphones and tablets.



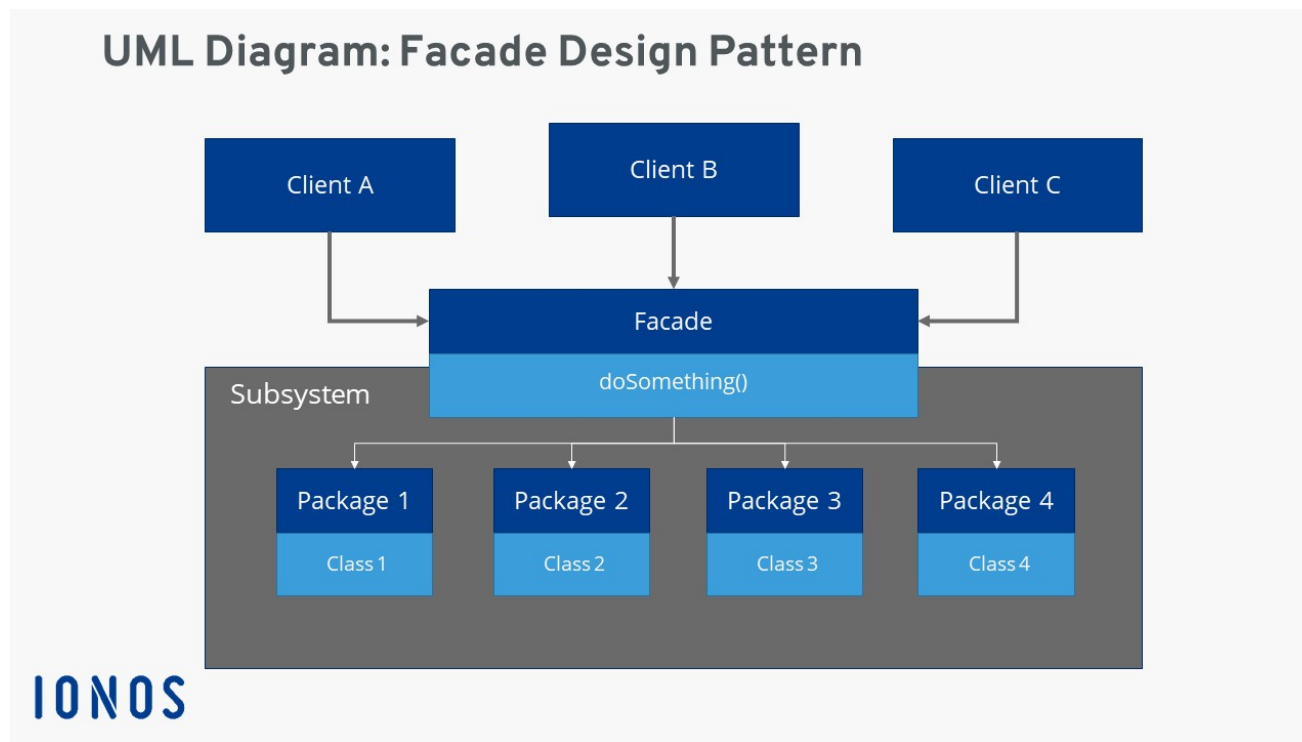
Software Architecture Basics

- Software architecture defines the structure and organization of a system.
- It focuses on components, interactions, and design decisions.
- Good architecture ensures scalability, maintainability, and performance.



Software Design Patterns

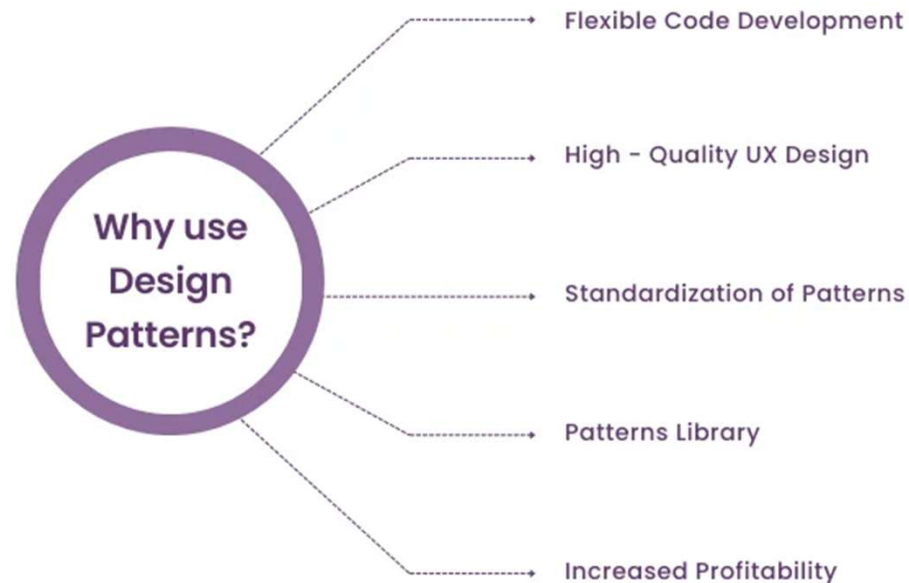
- Design patterns are reusable solutions to common software design problems.
- They are derived from industry best practices.
- Patterns improve code quality and consistency.



Benefits of Using Design Patterns

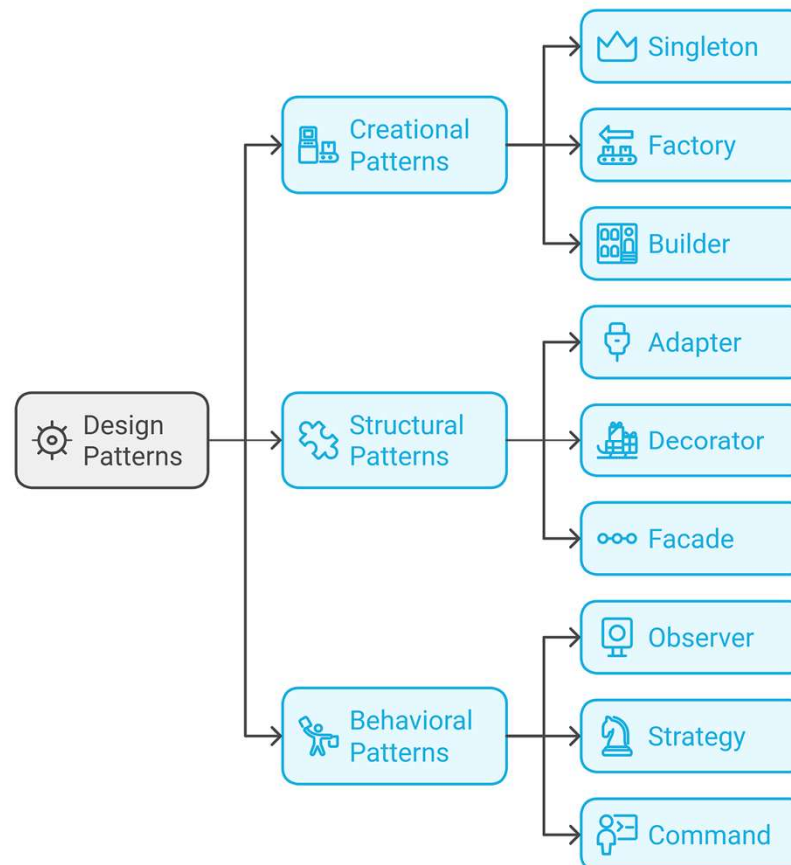
- Reduces development time.
- Improves maintainability and readability.
- Encourages standard design practices.

theknowledgeacademy



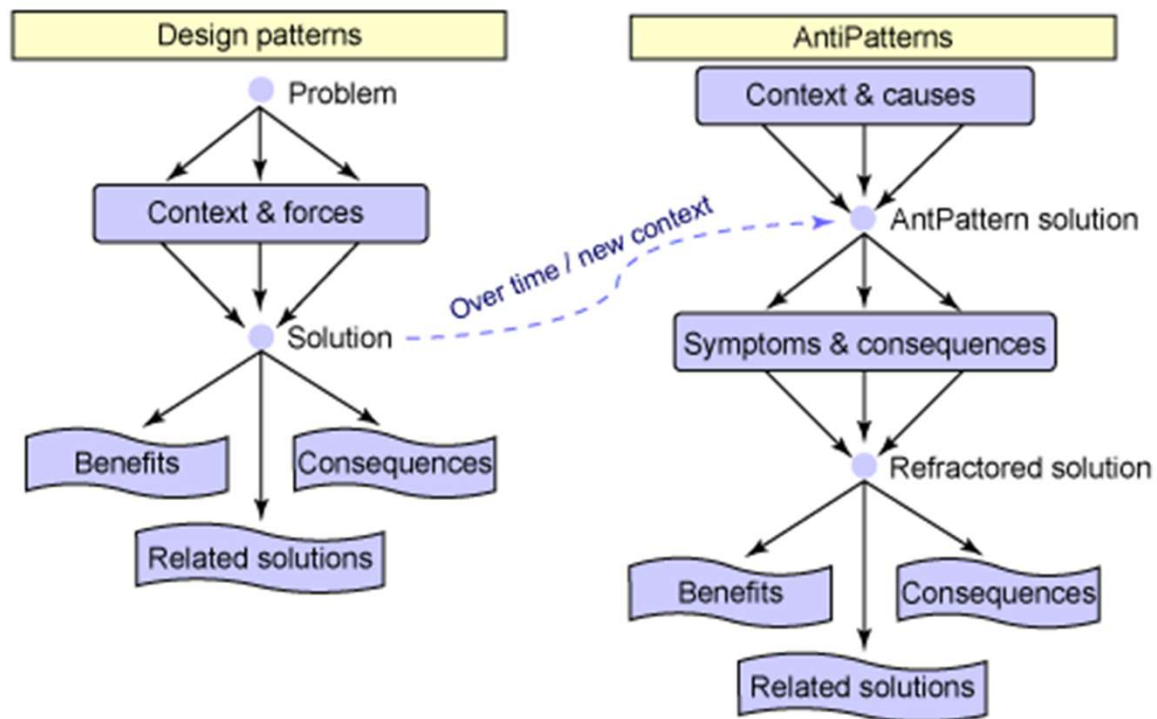
Common Design Patterns

- Creational Patterns: Singleton, Factory.
- Structural Patterns: Adapter, Decorator.
- Behavioral Patterns: Observer, Strategy.



Anti-Patterns

- Anti-patterns are poor solutions that appear effective initially.
- Examples include God Object and Spaghetti Code.
- They increase complexity and technical debt.



MVC Architecture

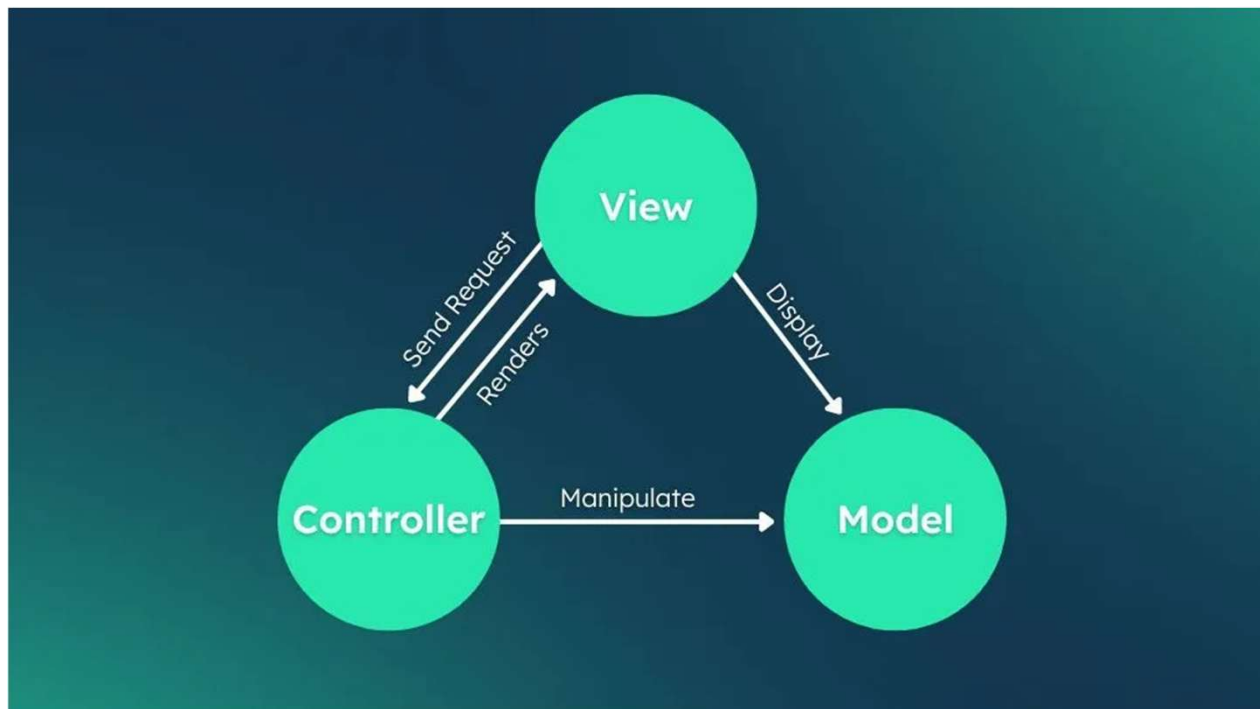
The Model-View-Controller (MVC) pattern separates an application into three components.

Model handles data and business logic.

View is responsible for the user interface.

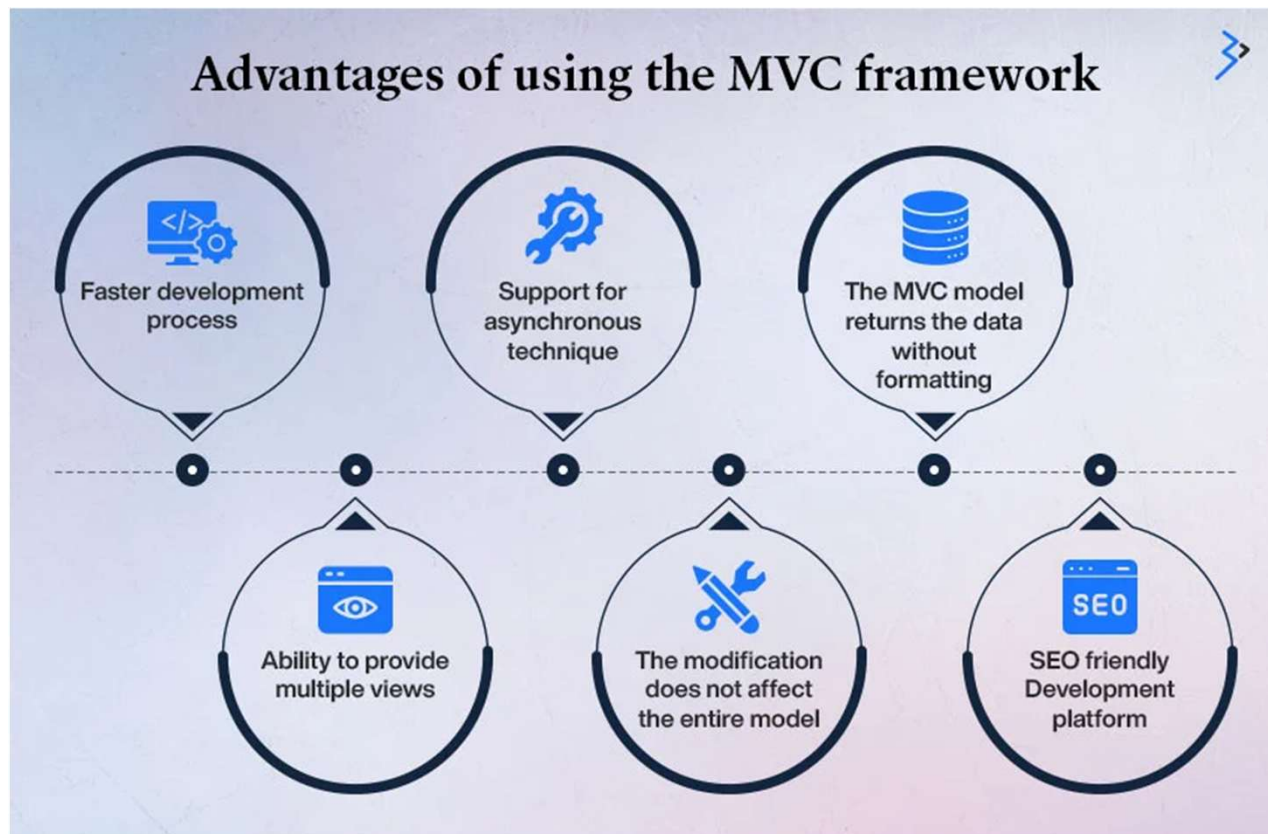
Controller handles user input and updates the Model and View.

This separation improves maintainability and testability.



Benefits and Variants of MVC

- Promotes separation of concerns.
- Allows parallel development.
- Variants include MVP and MVVM used in modern frameworks.



Service-Oriented Architecture (SOA)

SOA is an architectural style where applications are built as a collection of services.

Each service performs a specific business function.

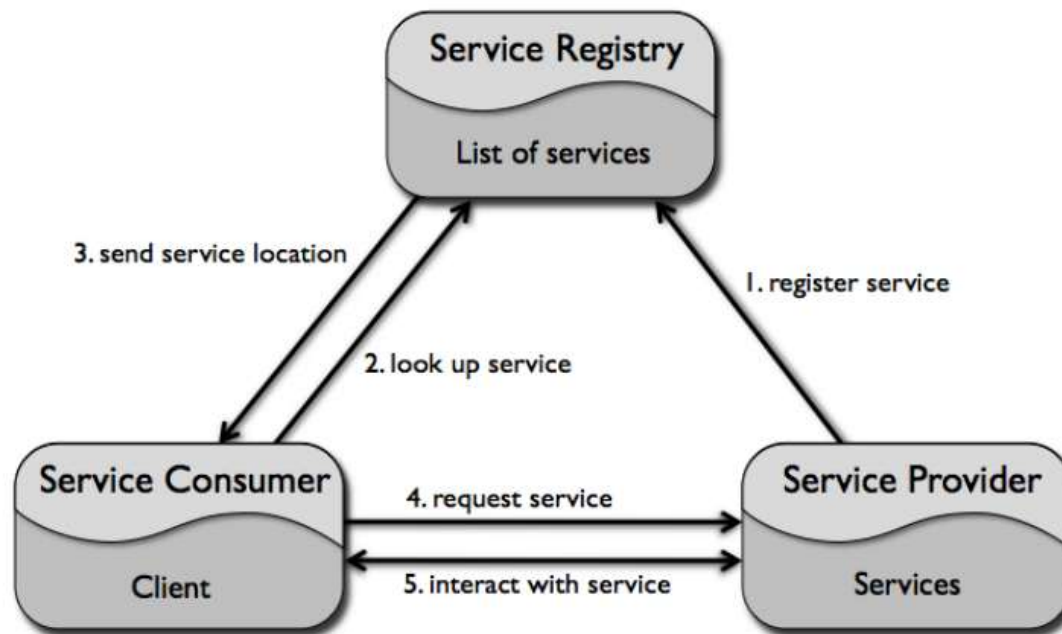
Services communicate using standard protocols.

SOA enables reuse and loose coupling.



Challenges in SOA

- Service governance complexity.
- Performance overhead due to communication.
- Managing service dependencies.



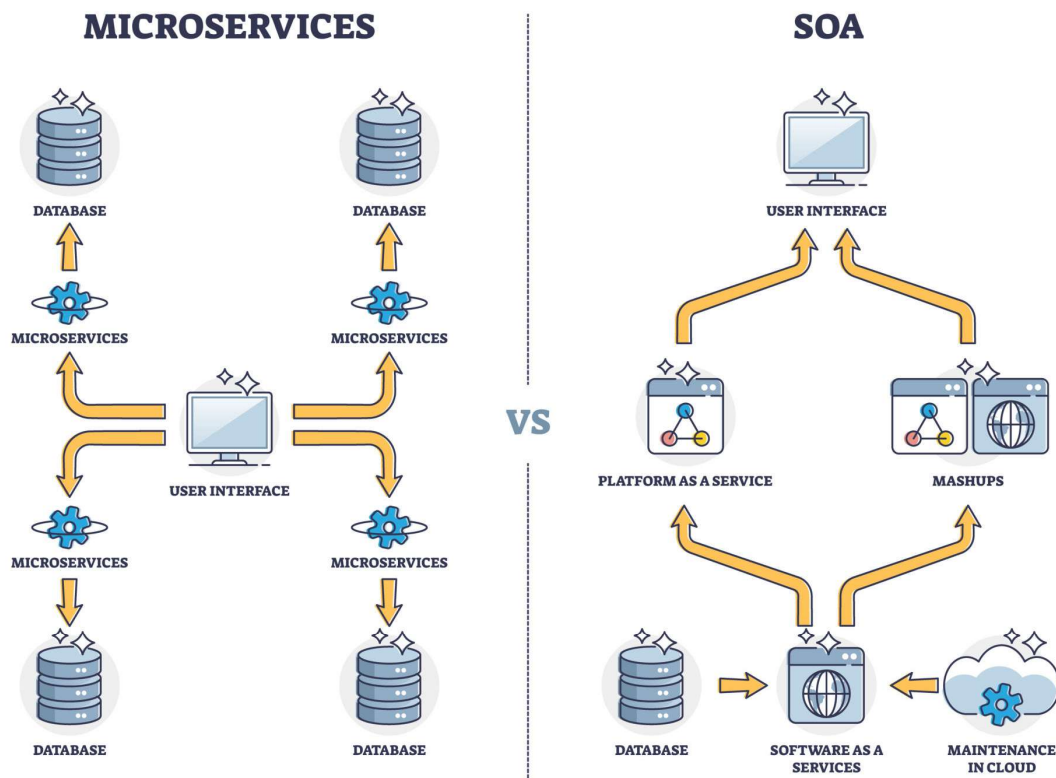
SOA vs Microservices

Microservices architecture decomposes applications into small, independent services.

Each service is independently deployable.

Microservices use lightweight communication mechanisms.

They offer better scalability than traditional SOA.



SOA vs Microservices – Comparison

- SOA uses centralized governance and coarse-grained services.
- Microservices use decentralized governance.
- Microservices scale and deploy independently.

